

■ 1.8.14 Advanced topic: Building Up Three-Dimensional Graphics

The functions `Plot3D` and `ListPlot3D` draw three-dimensional pictures of single *surfaces*. This section discusses how you can use *Mathematica*'s three-dimensional graphics primitives to build up more complicated three-dimensional pictures.

If you only use three-dimensional graphics to produce surfaces representing functions or lists of data, then you do not need to read this section. On the other hand, if you want to draw pictures of objects like polyhedra, this section will show you how to do it using three-dimensional graphics primitives.

<code>Graphics3D[list]</code>	representation of three-dimensional graphics
<code>Show[graphics]</code>	display graphics

Representation and display of three-dimensional graphics.

<code>Point[{x, y, z}]</code>	point with coordinates x, y, z
<code>Line[{{x₁, y₁, z₁}, {x₂, y₂, z₂}, ...}]</code>	line joining the points $(x_1, y_1), (x_2, y_2), \dots$
<code>Polygon[{{x₁, y₁, z₁}, {x₂, y₂, z₂}, ...}]</code>	filled polygon with the specified list of corners

<code>GrayLevel[i]</code>	display objects that follow with an intensity i
<code>RGBColor[r, g, b]</code>	display (if possible) with the specified red, green and blue color intensities
<code>PointSize[s]</code>	make points of size s
<code>Thickness[t]</code>	use thickness t for lines
<code>FaceForm[gfront, gback]</code>	show the fronts and backs of polygons in the specified <code>GrayLevel</code> or <code>RGBColor</code>
<code>EdgeForm[]</code>	do not draw lines at the edges of polygons
<code>EdgeForm[primitive]</code>	make polygon edges a particular <code>GrayLevel</code> or <code>RGBColor</code>

Three-dimensional graphics primitives.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

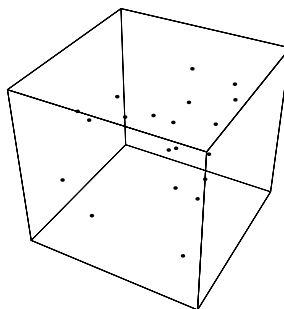
© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

This produces a list of 20 random points in three-dimensional space.

```
In[1]:= pts =  
Table[Point[{Random[], Random[], Random[]}], {20}] ;
```

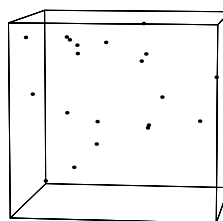
Here is a three-dimensional plot of the points.

```
In[2]:= Show[ Graphics3D [ pts ] ]
```



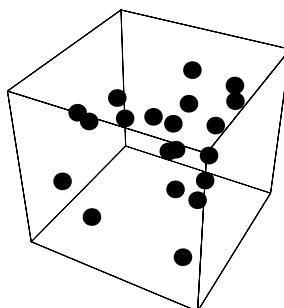
You can use `Show` to redraw the plot with different options.

```
In[3]:= Show[ %, ViewPoint -> {8, 1, 1} ]
```



This draws the plot with much larger points.

```
In[4]:= Show[ Graphics3D [ { PointSize[0.05], pts } ] ]
```

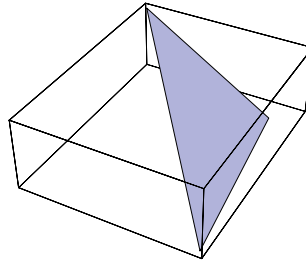


This defines a function that creates a random n -sided polygon.

```
In[5]:= ranpoly[n_] :=  
        Polygon[ Table[ Random[ ], {i, n}, {3} ] ]
```

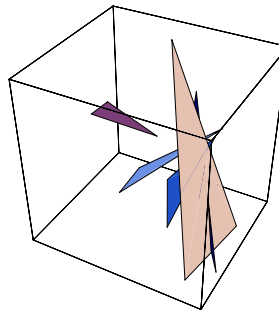
Here is a picture of a randomly-generated triangle.

```
In[6]:= Show[ Graphics3D[ ranpoly[3] ] ]
```



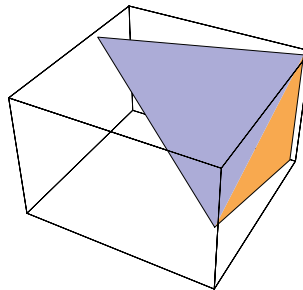
Here is a picture of 5 randomly-generated triangles. *Mathematica* hides the parts that are obscured.

```
In[7]:= Show[ Graphics3D[ Table[ranpoly[3], {5}] ] ]
```



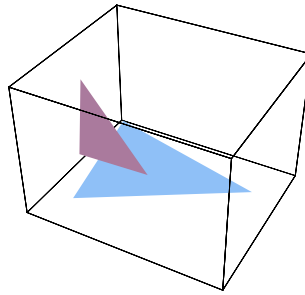
Mathematica assumes that all the polygons you give it are supposed to be planar. If you give a non-planar polygon, *Mathematica* will break the polygon into triangles, but not show any “seams” between the triangles.

```
In[8]:= Show[ Graphics3D[  
        Polygon[  
            {{3, 0, 2}, {0, 2, 3}, {3, 3, 1}, {3, 3, 3}}  
        ] ]]
```



Putting in `EdgeForm[ ]` gets rid of the lines at edges of polygons.

```
In[9]:= Show[ Graphics3D[ {EdgeForm[ ], ranpoly[3],
                             ranpoly[3]} ] ]
```



The way that shading and color are done in three-dimensional *Mathematica* graphics is flexible, but somewhat complicated.

There are two basic approaches. First, you can set `Lighting -> False`, and then the primitives `GrayLevel` and `RGBColor` to specify explicitly the shading of all the objects. Alternatively, you can set `Lighting -> True`, and generate polygon shadings from simulated illumination. You can then use `GrayLevel` and `RGBColor` to specify the *reflectance* of different polygons. The final shading of a polygon is determined by the *product* of its illumination and reflectivity. `Lighting` does not affect the shading of points and lines.

A final complication is that each polygon has two sides: a front and a back. By using the two arguments to the graphics primitive `FaceForm` you can specify different colors or reflectivities for the front and back. When you look at the *front* of the polygon, the corners, taken in the order that you specify them, appear in counter-clockwise order. Reversing the order of the corners interchanges the roles of front and back.

When you construct two-dimensional graphics, *Mathematica* always draws the objects you specify in the order you give them. Later objects are taken to obscure earlier ones. On the other hand, when you construct three-dimensional graphics, the order in which you specify different objects is essentially irrelevant. *Mathematica* will always order the objects so that objects “in front” obscure those “behind”.

The file `Polyhedra.m` contains lists of graphics primitives representing regular polyhedra.

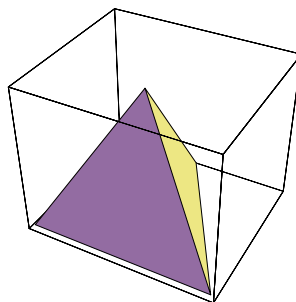
```
In[10]:= <<Polyhedra.m
Out[10]= Polyhedra`
```

Here are the graphics primitives for a tetrahedron centered at the origin. Note that each pair of faces share a common edge.

```
In[11]:= Tetrahedron[ ]
Out[11]= {Polygon[{{0., 0., 1.},
{-0.942809, 0., -0.333333},
{0.471405, -0.834546, -0.333333}}],
Polygon[{{0., 0., 1.},
{0.471405, 0.834546, -0.333333},
{-0.942809, 0., -0.333333}}],
Polygon[{{0., 0., 1.},
{0.471405, -0.834546, -0.333333},
{0.471405, 0.834546, -0.333333}}],
Polygon[{{-0.942809, 0., -0.333333},
{0.471405, 0.834546, -0.333333},
{0.471405, -0.834546, -0.333333}}]}
```

This displays the graphics primitives corresponding to a tetrahedron. Only two of the faces of the tetrahedron are visible from this angle.

```
In[12]:= Show[ Graphics3D[ % ] ]
```



A further detail concerns how three-dimensional pictures are built up on interactive display devices. You can choose whether *Mathematica* draws every polygon, but covers up ones that are not visible in the final picture, or whether it only draws polygons or parts of polygons that are going to be visible.

RenderAll->True	draw all polygons, starting from the back (default)
RenderAll->False	draw only those polygons or parts of polygons which are visible in the finished picture

An option for rendering three-dimensional pictures.

If you set **RenderAll->True**, then your pictures will always be drawn from back to front. All polygons will be rendered, but the ones at the back will be covered up

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

by ones in front. If you display a closed surface with `RenderAll->True`, you will see the insides of the surface while it is being drawn; the surface will typically be closed up only when the last polygons are added at the front of the picture.

With `RenderAll->False`, on the other hand, *Mathematica* renders only the polygons, or parts of polygons, that are visible in the final picture. As a result, you can never see the insides of closed surface, even while it is being drawn.

For reasonably simple pictures, without a large number of intersecting polygons, `RenderAll->False` will typically give you smaller POSTSCRIPT output. Particularly when there are a large number of intersecting polygons, however, `RenderAll->False` may take a very long time to run, and may give you larger POSTSCRIPT output. The final pictures you get are always independent of the setting you use for `RenderAll`; all that is affected is how your pictures are built up in time.